

Report — Version 2: Executor

Introduction

Version 2 introduces a new component: the **Executor**.

In Version 1, the Agent executed tools directly.

That approach worked, but it was not a clean separation of responsibilities.

Version 2 improves the architecture by moving execution logic into a dedicated component.

The main idea is:

```
Agent = coordinates  
Executor = validates and executes
```

Goal of Version 2

The goal of Version 2 is to separate coordination from execution.

The Agent should not be responsible for running tools directly.

Instead, it should delegate execution to the Executor.

This makes the architecture cleaner, easier to understand, and easier to extend.

What Changes from Version 1

In Version 1, the flow was:

```
User Request  
↓  
Router  
↓  
Parser  
↓  
Agent executes the tool directly  
↓  
Memory
```

In Version 2, the flow becomes:

User Request

↓

Router

↓

Parser

↓

Executor

↓

Memory

The main change is that the Agent no longer executes the selected tool by itself.

The Executor becomes responsible for that part.

Executor

The Executor is the main new component in Version 2.

It receives:

```
tool_name  
arguments
```

Then it performs two operations:

1. validate the arguments;
2. run the selected tool.

Example:

```
executor("addition", {"a": 2, "b": 3})
```

returns:

```
5
```

This makes the execution step explicit and separated from the Agent.

Argument Validation

The function:

```
arguments_are_valid(arguments)
```

checks whether the extracted arguments are usable.

For example:

```
{"a": 2, "b": 3}
```

is valid.

But:

```
{"a": 2, "b": None}
```

is not valid.

This function belongs inside the Executor section because validation is part of the execution process.

The Executor should not run a tool if the required arguments are missing.

Updated Role of the Agent

In Version 2, the Agent becomes simpler.

The Agent now:

1. receives the user request;
2. uses the Router to select the tool;
3. uses the Parser to extract arguments;
4. sends the tool and arguments to the Executor;
5. stores the final state in memory.

The Agent no longer needs to know how the tool is executed.

It only coordinates the flow.

This is a major architectural improvement.

Full Flow Example

User request:

What is 2 + 3?

The Router selects:

"addition"

The Parser extracts:

{"a": 2, "b": 3}

The Executor runs:

addition(a=2, b=3)

The result is:

5

The final state becomes:

```
{
  "request": "What is 2 + 3?",
  "action": "addition",
  "arguments": {"a": 2, "b": 3},
  "result": 5
}
```

Notebook Structure

The correct structure for Version 2 is:

```
1. Imports
2. Tools
3. Parser
4. Router
5. Executor
6. Agent
7. Tests
Notes
```

The Executor section should contain:

```
arguments_are_valid()
executor()
```

The Agent section should contain only the orchestration logic.

This makes the notebook clearer and more professional.

Tests

The test cases verify that the agent still works after introducing the Executor.

For example:

```
agent("What is 2 + 3?")
```

should return a result of:

```
5
```

Another important test is:

```
agent("Hello, what is 2 + 3?")
```

This checks that the Router prioritizes the main intent, which is addition, instead of treating the request as a greeting.

Another test is:

```
agent("Hello, my name is luca.")
```

This checks that the greeting tool receives the extracted name and formats it correctly:

```
Hello Luca, nice to meet you!
```

What Version 2 Teaches

Version 2 teaches an important software design principle:

each component should have one clear responsibility

The Agent should not do everything.

Execution is important enough to become a separate component.

This separation makes the system easier to maintain and prepares it for future versions.

Limitations

Version 2 improves the architecture, but some limitations remain:

- the Parser is still rigid;
- the Router is still rule-based;
- memory is still a simple list;
- there is no Planner;
- there is no Tool Registry;
- there is no LLM;
- the agent still executes only one action at a time.

These limitations will be solved step by step in later versions.

Conclusion

Version 2 is the first real architectural improvement of the project.

The Agent no longer executes tools directly.

Instead, it delegates execution to the Executor.

The Executor validates the parsed arguments and runs the selected tool.

This makes the project cleaner, more modular, and closer to a professional agent architecture.

The next step is Version 3, where the Parser will become generic.